



deti

universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

CPU Scheduling

Real-Time Embedded Systems (#41505)

Paulo Pedreiras, pbrp@ua.pt
Department of Electronics, Telecommunications and Informatics
University of Aveiro
Academic year 2025/2026

Outline

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling
- Multi-Processor Scheduling
- Real-Time CPU Scheduling
- Operating Systems Examples

Basic Concepts

- What is scheduling?
- A dictionary is always a good starting point:

schedule 2 of 2 **verb**

scheduled; scheduling

transitive verb

1 : to appoint, assign, or designate for a fixed time

2 a : to place in a schedule

b : to make a schedule of

scheduler **noun**

Source:

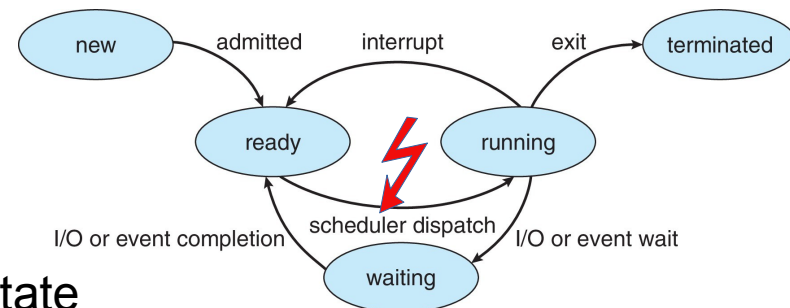
Merriam-Webster

<https://www.merriam-webster.com/dictionary/scheduling>

- Thus, CPU scheduling is all about to map processes/tasks/threads into CPUs over time!
 - ~ Simple to define, but extremely hard to make “efficiently”, as we will see!

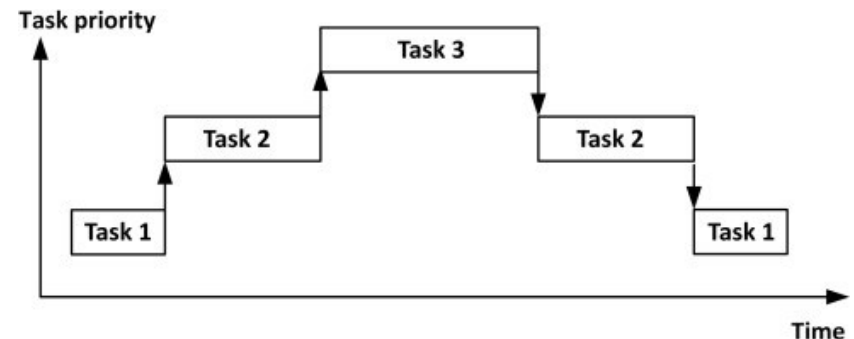
CPU Scheduler

- At runtime, the **CPU scheduler** selects one of the processes in the ready queue, and allocates a CPU core to it
 - The ready queue can be sorted in various ways
- CPU scheduling decisions may take place when a process:
 - 1) Switches from running to waiting state
 - 2) Switches from running to ready state
 - 3) Switches from waiting to ready
 - 4) Terminates
- For events 1 and 4 there is no choice:
 - ✎ A new process (if one exists in the ready queue) must be selected for execution.
- For situations 2 and 3, however, selecting a new process to execute is not mandatory.



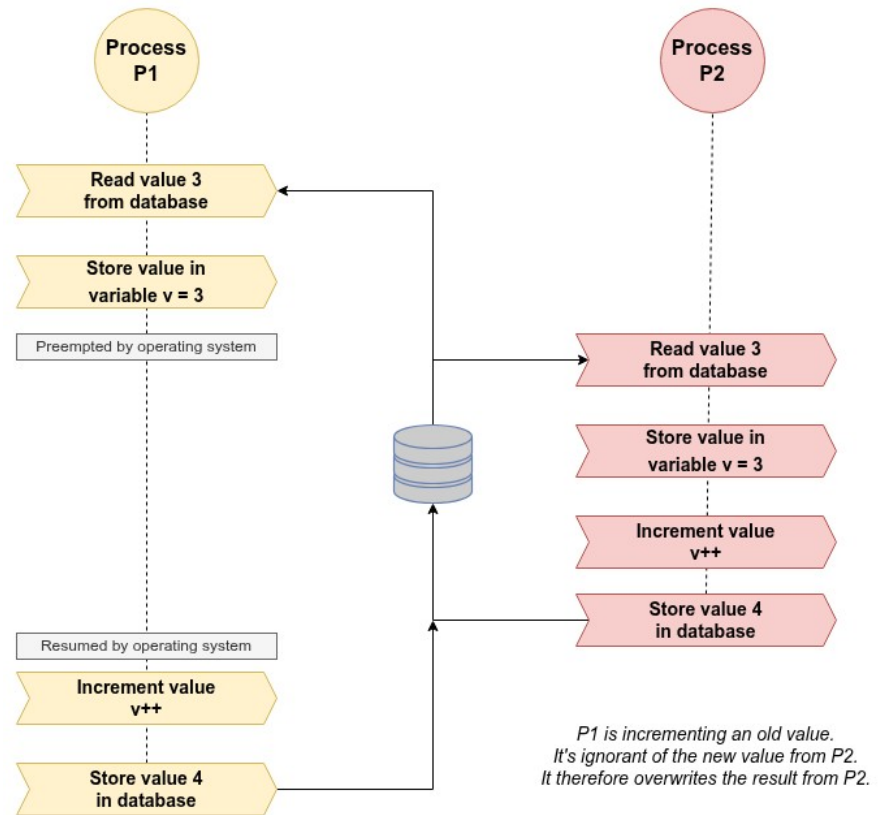
Preemptive and Nonpreemptive Scheduling

- When scheduling takes place only when the running process blocks or terminates (scenarios 1 and 4), the scheduling scheme is **nonpreemptive**. Otherwise, it is **preemptive**.
- **Nonpreemptive** scheduling
 - ~ Once a process starts executing it keeps the CPU until terminating or switching to the waiting state.
- **Preemptive** scheduling
 - ~ A running process can be moved to the ready queue to allow the execution of others (somehow more “important”) processes
- **Virtually all modern operating systems** including Windows, MacOS, Linux, and UNIX use **preemptive** scheduling algorithms.
 - ~ Improves response time and interactivity, ..., at expenses of resources (CPU and memory)



Preemptive Scheduling and Race Conditions

- **Preemptive scheduling can result in race conditions when data are shared among several processes.**
- Consider the case of two processes that share data. While one process is updating the data, it is preempted so that the second process can run. The second process then reads and updates the same data, which are in an inconsistent state.
- This issue will be explored in a future class



Source:
<https://devopedia.org/images/article/14/2173.1597585204.svg>

Scheduling Criteria

Multiple criteria can be used to design a “good” scheduler

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – maximize the number of processes that complete their execution per time unit
- **Turnaround time** – minimize the amount of time to execute a particular process
- **Waiting time** – minimize the amount of time a process has been waiting in the ready queue
- **Response time** – minimize the amount of time it takes from when a request was submitted until the first response is produced

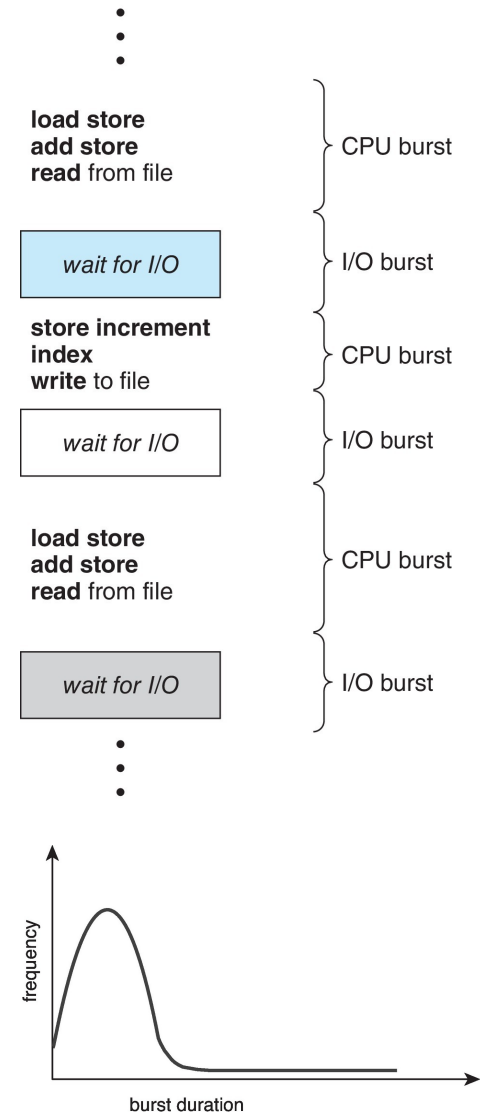
~ The more common requirement in real-time systems / RTOS, as will see

Scheduling Algorithms/Policies

- The scheduler applies a given algorithm to decide which (ready) task should be granted to the CPU
- This **Scheduling Algorithm** uses some parameters (static, dynamic or both) to sort the ready queue
- Different scheduling algorithms optimize different objectives
 - ~ E.g. minimize waiting time or response time, provide fairness, ...
- Lets see some examples, but first an important definition ...

CPU Burst / Execution time

- Typical pattern: CPU–I/O Burst Cycle
 - └ Process execution consists of a **CPU burst** followed by **I/O burst**
 - └ CPU burst distribution is of main concern
- Other common pattern is when tasks run an instance (job) from time to time, without I/O wait time during each job - **common in RT systems**
 - └ In this case the time each iteration/job takes is called execution time



First- Come, First-Served (FCFS) Scheduling

<u>Process</u>	<u>Execution/Burst Time</u>
----------------	-----------------------------

P_1	24
-------	----

P_2	3
-------	---

P_3	3
-------	---

- Suppose that the processes arrive in the order: P_1, P_2, P_3
The Gantt Chart for the schedule is:



- Waiting time: $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

FCFS Scheduling (Cont.)

Suppose that the processes arrive in the order:

$$P_2, P_3, P_1$$

- The Gantt chart for the schedule is:



- Waiting time: $P_1 = 6; P_2 = 0; P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case
- **Convoy effect** - short process behind long process
 - Consider one CPU-bound and many I/O-bound processes

Shortest-Job-First (SJF) Scheduling

- Associate with each process the length of its burst/execution time
 - Use these lengths to schedule the process with the shortest time
- In theory SJF is **optimal**
 - Gives minimum average waiting time for a given set of processes
- Preemptive version called **shortest-remaining-time-first**
- How do we determine the length of the next CPU burst/execution time?
 - Could ask the user
 - Estimate
 - No exact value known in advance, so in practice you lose optimality!

Example of SJF

Process Execution/Burst Time

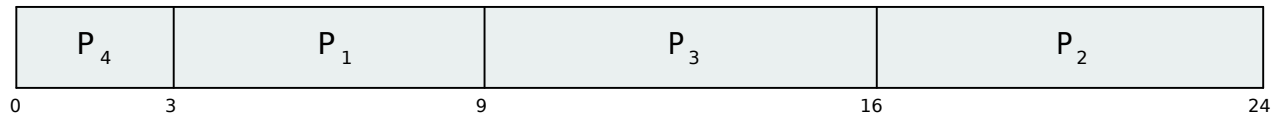
P_1 6

P_2 8

P_3 7

P_4 3

- SJF scheduling chart

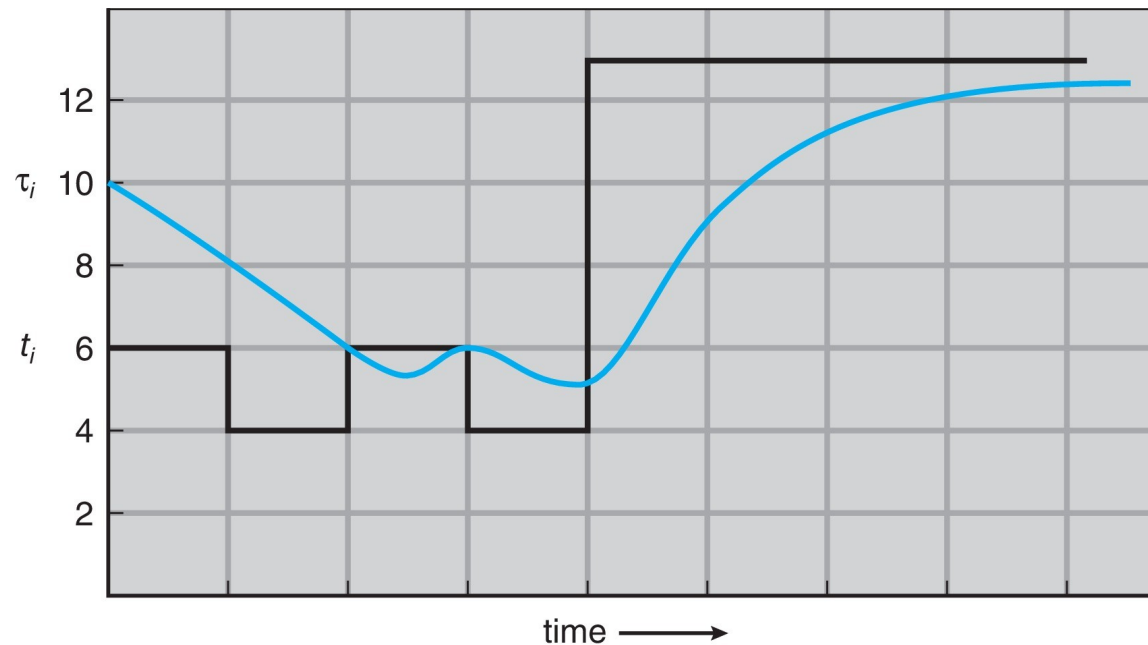


- Average waiting time = $(3 + 16 + 9 + 0) / 4 = 7$

Determining Length of Next CPU Burst

- Can only **estimate** the length – should be similar to the previous one
 - Then pick process with shortest predicted next CPU burst
- Can be done e.g. by using the length of previous CPU bursts, using exponential averaging
 1. t_n = actual length of n^{th} CPU burst
 2. τ_{n+1} = predicted value for the next CPU burst
 3. α , $0 \leq \alpha \leq 1$
 4. Define:
$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n.$$
- Commonly, α set to $\frac{1}{2}$

Prediction of the Length of the Next CPU Burst



CPU burst (t_i)	6	4	6	4	13	13	13	...	
"guess" (τ_i)	10	8	6	6	5	9	11	12	...

Shortest Remaining Time First Scheduling

- Preemptive version of SJF
- Whenever a new process arrives in the ready queue, the decision on which process to schedule next is redone using the SJF algorithm.
- Is SRT more “optimal” than SJF in terms of the minimum average waiting time for a given set of processes?

Example of Shortest Remaining Time First

Now we add the concepts of varying arrival times and preemption to the analysis

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

■ *Preemptive* SJF Gantt Chart



- Average waiting time = $[(10-1)+(1-1)+(17-2)+(5-3)]/4 = 26/4 = 6.5$

Round Robin (RR)

- Each process gets a small unit of CPU time (**time quantum** q), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.
- Timer interrupts every quantum to schedule next process
- Performance
 - q “large” $\rightarrow$ FIFO (FCFS)
 - q “small” $\rightarrow$ RR
- Note that q must be large with respect to context switch time, otherwise overhead is too high

Example of RR with Time Quantum = 4

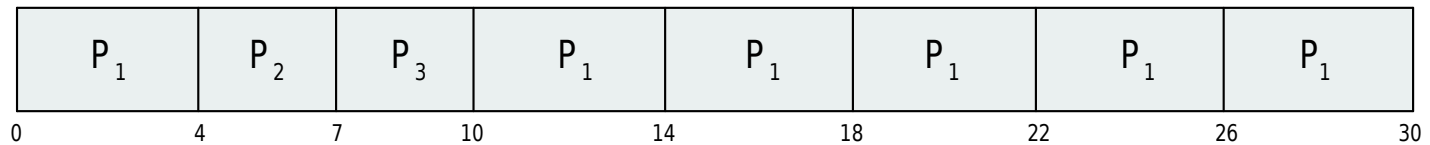
Process Burst Time

P_1 24

P_2 3

P_3 3

- The Gantt chart is:



- Typically, higher average turnaround than SJF, but better **response**
- q should be large compared to context switch time
 - q usually 10 milliseconds to 100 milliseconds,
 - Context switch < 10 microseconds

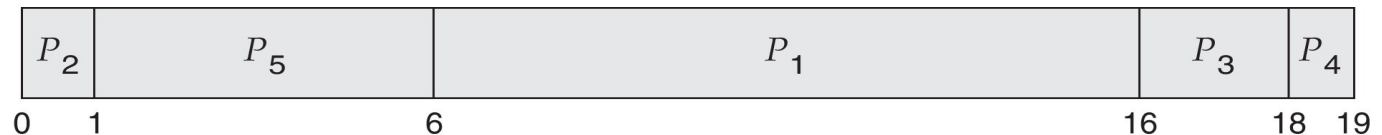
Priority Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority. Relation between number and priority level varies; some OSs make smaller values correspond to higher priorities, others the opposite.
 - Preemptive
 - Nonpreemptive
- SJF is **dynamic priority** scheduling where priority is the inverse of predicted next CPU burst time
- It is also possible to assign a **static/fixed priority** to a process (common in RT systems)
- Problem → **Starvation** – low priority processes may never execute
- Solution → **Aging** – as time progresses increase the priority of the process

Example of Priority Scheduling

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u> (Higher number - > smaller priority)
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

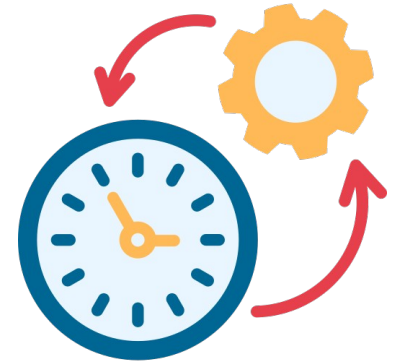
- Priority scheduling Gantt Chart



- Average waiting time = 8.2

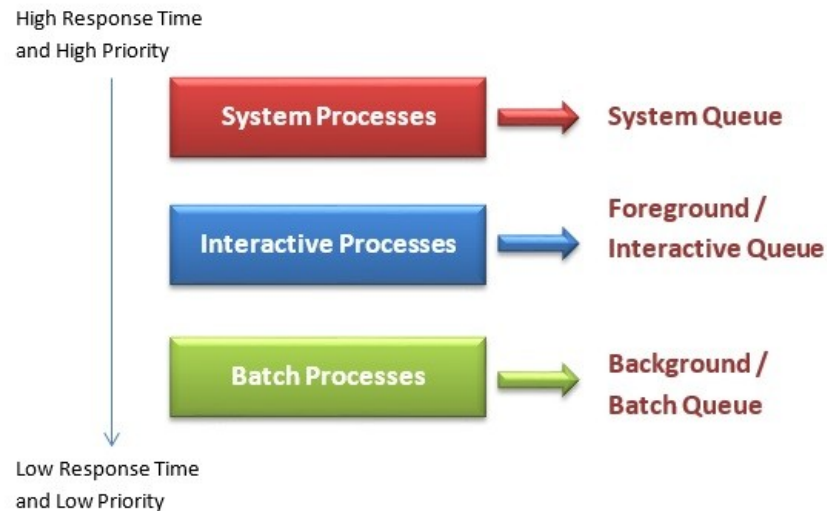
Priority Scheduling

- Priority scheduling at the core of real-time systems
 - ~ Allows to guarantee that critical tasks are executed on time.
- By giving precedence to high-priority processes, it reduces response latency and prevents deadline misses.
- This ensures system predictability, reliability, and stable performance under time constraints.
- Priority scheduling theory will be addressed in detail in the second part of the course unit



Multilevel Queue

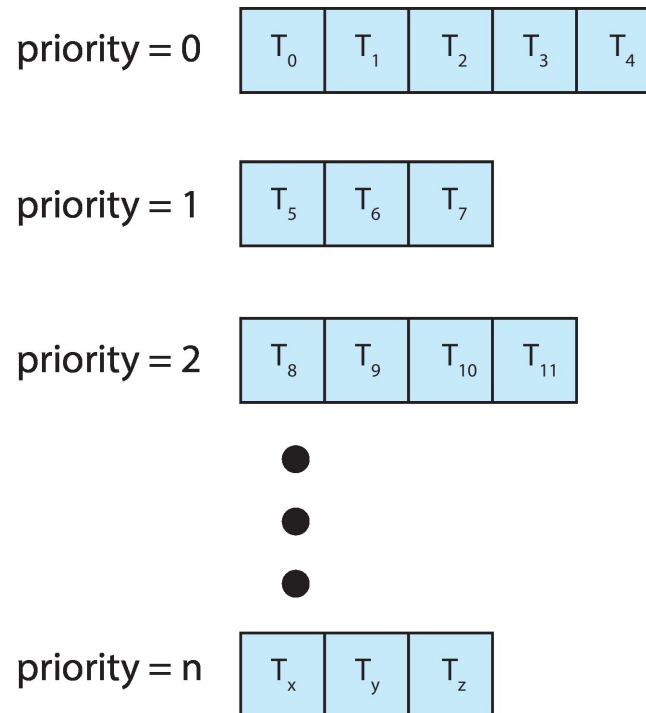
- The ready queue consists of multiple queues
- A Multilevel Queue scheduler is defined by the following parameters:
 - Number of queues
 - Scheduling algorithms for each queue
 - Method used to determine which queue a process will enter when that process needs service
 - Scheduling among the queues



Source:
https://www.tutorialspoint.com/operating_system/os_multilevel_queue_scheduling.htm

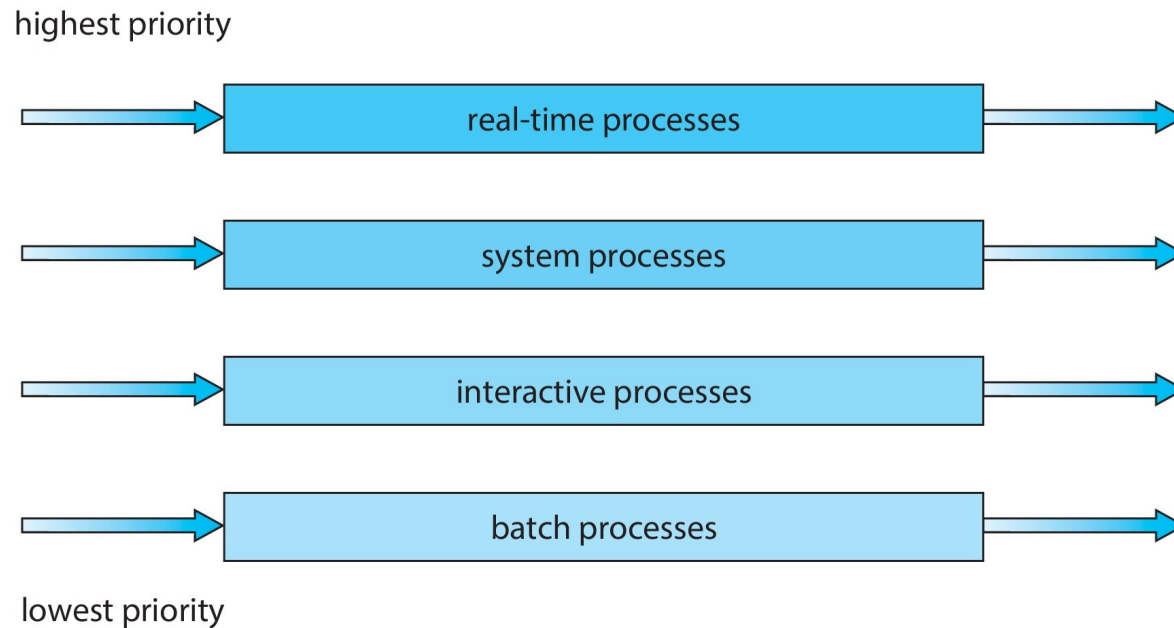
Multilevel Queue

- With priority scheduling, have separate queues for each priority.
- Schedule the process in the highest-priority queue!



Multilevel Queue

- Prioritization based upon process type/class

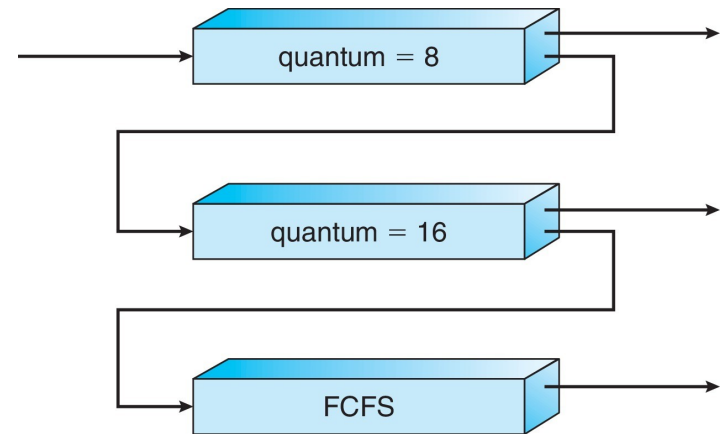


Multilevel Feedback Queue

- A process can move between the various queues.
- Multilevel-feedback-queue scheduler defined by the following parameters:
 - Number of queues
 - Scheduling algorithms for each queue
 - Method used to determine when to upgrade a process
 - Method used to determine when to demote a process
 - Method used to determine which queue a process will enter when that process needs service
- Aging can be implemented using multilevel feedback queue

Example of Multilevel Feedback Queue

- Three queues:
 - Q_0 – RR with time quantum 8 milliseconds
 - Q_1 – RR time quantum 16 milliseconds
 - Q_2 – FCFS
- Scheduling
 - A new process enters queue Q_0 which is served in RR
 - ▶ When it gains CPU, the process receives 8 milliseconds
 - ▶ If it does not finish in 8 milliseconds, the process is moved to queue Q_1
 - At Q_1 job is again served in RR and receives 16 additional milliseconds
 - ▶ If it still does not complete, it is preempted and moved to queue Q_2
 - It is possible to use Aging mechanisms to move processes that waiting for too long to a higher priority queue



Thread Scheduling

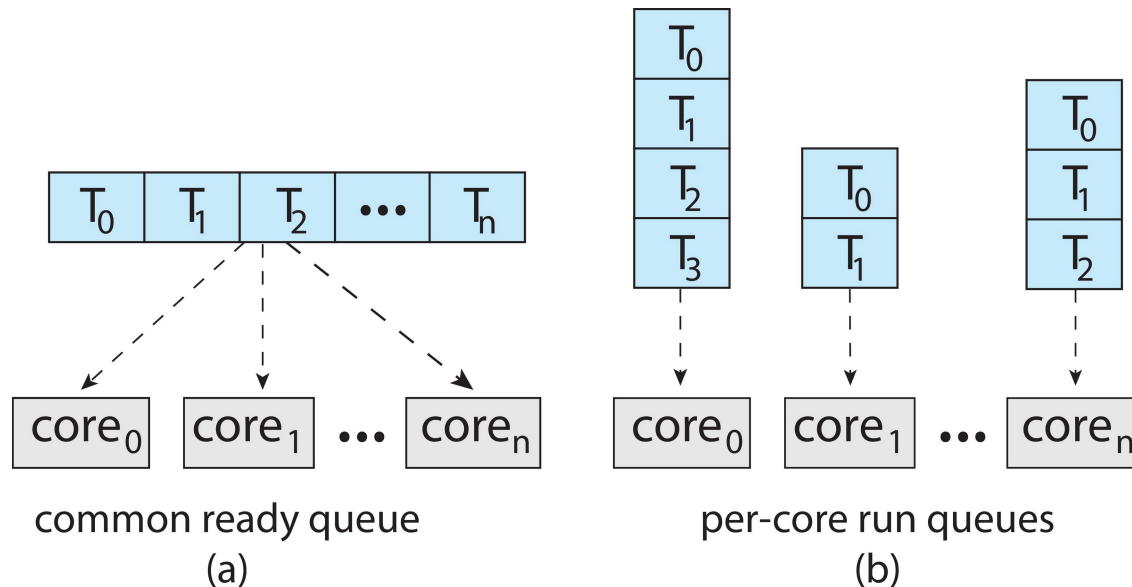
- Distinction between user-level and kernel-level threads
- User-level threads
 - ~ The threads are managed by the thread library – not individually
 - ~ Application can influence scheduling only among its own threads
 - ~ Known as **process-contention scope (PCS)** since scheduling competition is within the process
- Kernel-level threads
 - Threads scheduled onto available CPU(s) is **system-contention scope (SCS)** – competition among all threads in system
 - **Nowadays more widely supported (e.g. Linux and macOS only support PTHREAD_SCOPE_SYSTEM).**

Multiple-Processor Scheduling

- CPU scheduling more complex when multiple CPUs are available
- Multiprocess architectures include:
 - Multicore CPUs
 - Multithreaded cores
 - (N)UMA systems / (Non-)Uniform Memory Access
 - Heterogeneous multiprocessing

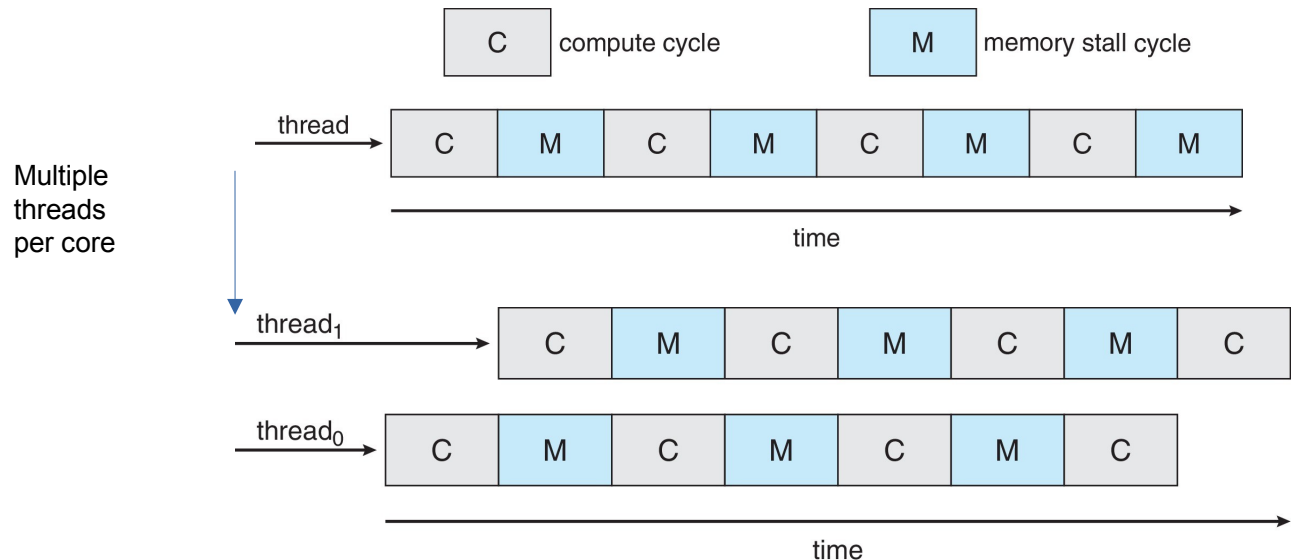
Multiple-Processor Scheduling

- Symmetric multiprocessing (SMP) is a computer architecture where two or more identical processors share the same main memory and I/O, and are controlled by a single OS instance.
- All threads may be in a common ready queue (a)
- Each processor may have its own private queue of threads (b)



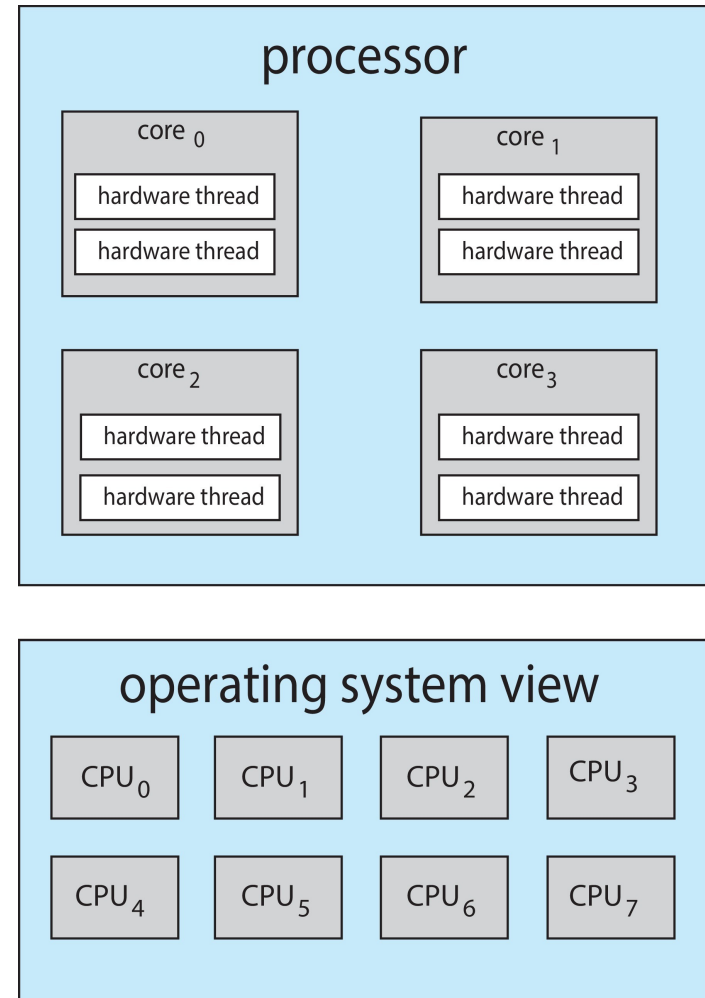
Multicore Processors

- Recent trend to place multiple processor cores on same physical chip
- Faster and consumes less power
- **Multiple threads per core also growing**
 - ~ E.g. Intel Hyper-Threading allows a single physical core to present itself as two logical processors, each capable of running a separate thread.
 - ~ Takes advantage of memory stall to make progress on another thread while memory retrieve happens
 - If each core supports more the 1 hardware thread, when one thread has a memory stall, switch to another thread!



Multithreaded Multicore System

- **Chip-multithreading (CMT)** assigns each core multiple hardware threads. (Intel refers to this as **hyperthreading**.)
- On a quad-core system with 2 hardware threads per core, the operating system sees 8 logical processors.



Multiple-Processor Scheduling – Load Balancing

- If SMP, need to keep all CPUs loaded for efficiency
- **Load balancing** attempts to keep workload evenly distributed
- **Push migration** – periodic task checks load on each processor, and if found pushes task from overloaded CPU to other CPUs
- **Pull migration** – idle processors pulls waiting task from busy processor

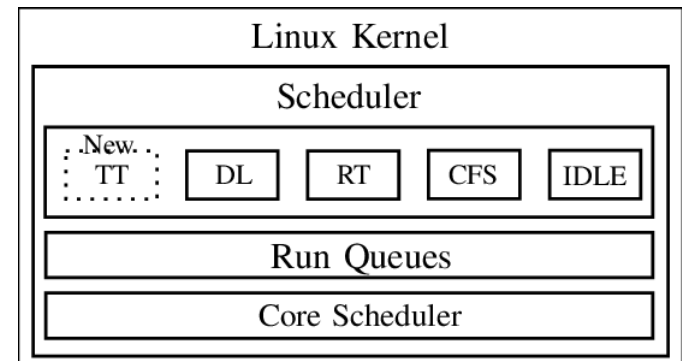
Multiple-Processor Scheduling – Processor Affinity

- When a thread has been running on one processor, the cache contents of that processor stores the memory accesses by that thread.
- We refer to this as a thread having affinity for a processor (i.e., “processor affinity”)
- Load balancing may affect processor affinity as a **thread may be moved** from one processor to another to balance loads, yet that **thread loses the contents of what it had in the cache** of the processor it was moved off of.
 - ~ **Soft affinity** – the operating system attempts to keep a thread running on the same processor, but no guarantees.
 - ~ **Hard affinity** – allows a process to specify a set of processors it may run on.

Linux Scheduling

Linux supports several scheduling classes, including

- CFS (Completely Fair Scheduler) – Default for normal processes
 - ~ Tries to share CPU time “fairly” among tasks.
 - ~ Priority is influenced by nice values (-20 = highest priority, +19 = lowest).
- RT (Real-Time) Scheduling – For time-critical tasks
 - ~ Includes two policies:
 - SCHED_FIFO (First-In, First-Out): Runs tasks in strict priority order until they block or finish.
 - SCHED_RR (Round-Robin): tasks at the same priority get time slices
 - ~ Priorities range from 1 to 99 (higher = more important).
 - ~ Can preempt “normal tasks” (e.g. CFS tasks).
 - ~ Useful for embedded systems, multimedia, or control systems.
- Idle Scheduling (SCHED_IDLE)
 - ~ Very low priority, only runs when nothing else is runnable.



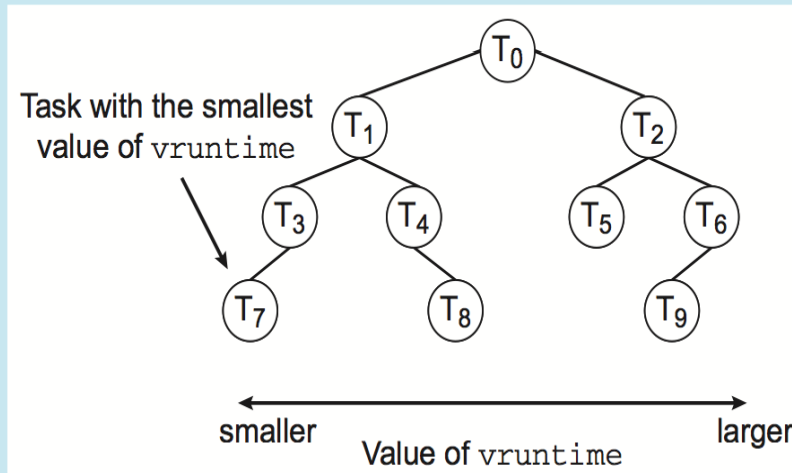
Linux Scheduling

- CFS – a few details

- ~ Quantum calculated based on **nice value** from -20 to +19
 - Lower value is higher priority
 - Calculates **target latency** – interval of time during which task should run at least once
 - Target latency can increase if the number of active tasks increases
- CFS scheduler maintains per task **virtual run time** in variable **vruntime**
 - Associated with decay factor based on priority of task – lower priority is higher decay rate
 - Normal default priority yields virtual run time = actual run time
- To decide next task to run, scheduler picks task with lowest virtual run time

CFS Performance

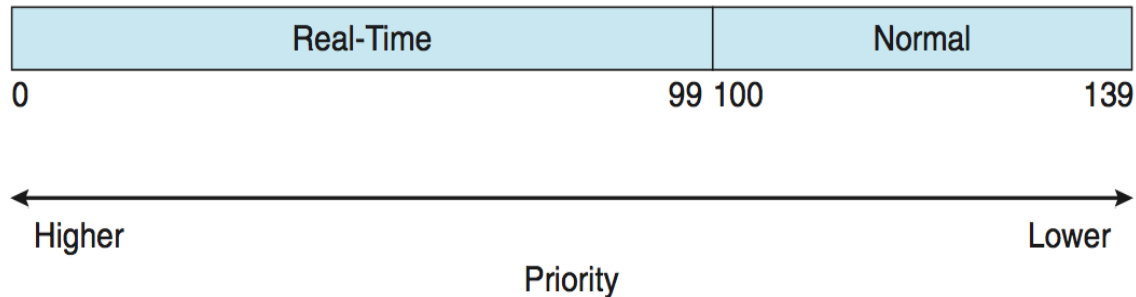
The Linux CFS scheduler provides an efficient algorithm for selecting which task to run next. Each runnable task is placed in a red-black tree—a balanced binary search tree whose key is based on the value of `vruntime`. This tree is shown below:



When a task becomes runnable, it is added to the tree. If a task on the tree is not runnable (for example, if it is blocked while waiting for I/O), it is removed. Generally speaking, tasks that have been given less processing time (smaller values of `vruntime`) are toward the left side of the tree, and tasks that have been given more processing time are on the right side. According to the properties of a binary search tree, the leftmost node has the smallest key value, which for the sake of the CFS scheduler means that it is the task with the highest priority. Because the red-black tree is balanced, navigating it to discover the leftmost node will require $O(\lg N)$ operations (where N is the number of nodes in the tree). However, for efficiency reasons, the Linux scheduler caches this value in the variable `rb_leftmost`, and thus determining which task to run next requires only retrieving the cached value.

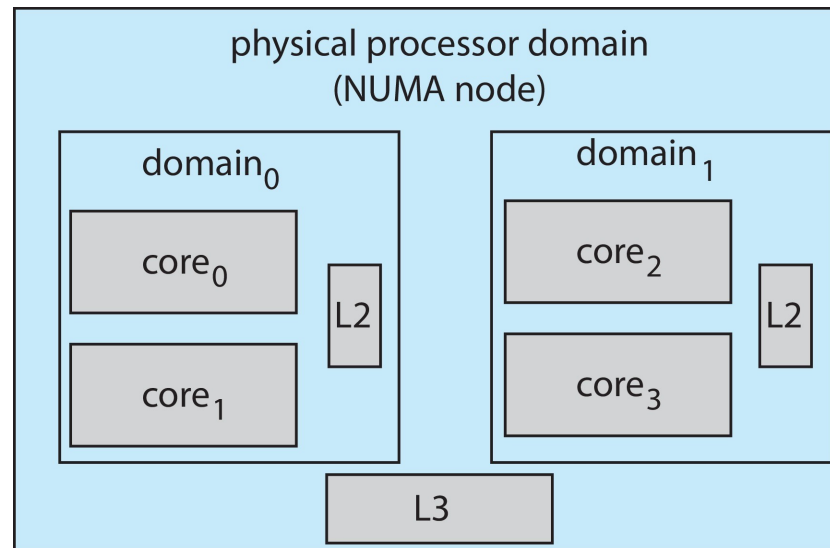
Linux Scheduling (Cont.)

- Real-time scheduling according to POSIX.1b
 - Real-time tasks have static priorities
- Real-time plus normal map into global priority scheme
- Nice value of -20 maps to global priority 100
- Nice value of +19 maps to priority 139



Linux Scheduling (Cont.)

- Linux supports load balancing, but is also NUMA-aware.
- **Scheduling domain** is a set of CPU cores that can be balanced against one another.
- Domains are organized by what they share (i.e., cache memory.) Goal is to keep threads from migrating between domains.



Real-Time Scheduling in Linux

API call:

int sched_setscheduler(pid_t pid, int policy, const struct sched_param *param);
sets the scheduling policy (SCHED_FIFO, SCHED_RR) and priority of a process or thread.

```
...
int main() {
    struct sched_param param;
    param.sched_priority = 20; // valid range depends on policy

    if (sched_setscheduler(0, SCHED_RR, &param) == -1) {
        perror("sched_setscheduler");
        return 1;
    }

    printf("Policy set to SCHED_RR with priority %d\n", param.sched_priority);
    while (1); // keep running
}
```

It is also possible to run a “normal” application with a give RT scheduler and priority

chrt - manipulate the real-time attributes of a process

E.g.: \$sudo chrt -f 50 ./my_program

Note: requires SU privileges

Real-Time Scheduling in Linux

```
...
int main() {
    pthread_t tid;
    pthread_attr_t attr;
    struct sched_param param;

    pthread_attr_init(&attr);

    // Explicitly set scheduling policy to RR
    pthread_attr_setschedpolicy(&attr, SCHED_RR);

    // Set priority (must be within policy's allowed range)
    param.sched_priority = 10;
    pthread_attr_setschedparam(&attr, &param);

    // Ensure scheduling attributes are applied
    pthread_attr_setinheritsched(&attr, PTHREAD_EXPLICIT_SCHED);
    if (pthread_create(&tid, &attr, worker, (void*)1L) != 0) {
        perror("pthread_create");
        exit(1);
    }
    pthread_join(tid, NULL);
    return 0;
}
```



- Questions?

Acknowledgment:

**This presentation is based on the companion materials provided with the book
Operating System Concepts – 10h Edition, by Silberschatz, Galvin and Gagne**